this is front page stuff

Acknowledgements

This template was developed and compiled from multiple sources. It owes its origins most firmly to a mil-spec template that I found on line years ago. That template contained several humorous lines about how boring specifications are and that cleanliness is next to Godliness. I had never laughed before when reading or writing a specification. I liked the author's approach.

The source of that original template is lost but the framework reappeared several years later as a Small Satellite requirements template but I am sure that it was a modification of the original. Next, this template also owes much of its structure to the brilliant book by Phillip Koopman 'Better Embedded Software' a book that describes a development process that is useful for a whole lot more than software.

Much of the current structure and helpful comments are the work of my colleague Jolynne Berrett. She has made the design process much less intimidating for Senior Design students. They tend to lose that 'deer-in-headlights' look after they realize that Jolynne will help them bear the heavy load of project documentation.

Preface

Requirements documents spring out of military/aerospace methods and are part of a formal process intended to minimize needed redesign. These systems design methods are heavily front-loaded requiring extensive pre-design effort. When properly exercised these methods can, and do, produce safe, effective systems that are on time and budget. Requirements documents help with the so-called concurrent engineering process in that all aspects of the project are specified at the beginning of the project down to and including the kind of box that the system is going to be put in. Concurrent engineering lessens the chances of unpleasant surprises late in the program. Hearing things like, "What do you mean it has to fit in a two inch cube?", six months into a program is frustrating and expensive. There are less formal methodologies that demand less up front work, but while it is easy to relax the formal it is often hard to go the other way.

Software developers often advocate design through methods like Extreme Programming or Agile. Some pieces of these methods are quite useful, e.g. users stories, but as a crusty old design engineer I do not like an approach that uses near constant redesign as a design methodology. When push comes to shove and lives are at stake, engineers will almost always choose a formal systems design methodology.

Many people say that this formal style of engineering development is passé and that newer, speedier, more flexible methods should be taught and used. And yet they unknowingly depend on these formal techniques every time they step onto an airplane. To those who pine for the less formal methods I always ask the following question:

> *Would you fly on an airplane designed by the methods you advocate?*

The answer is seldom yes.

How to Use This Document

This document is divided into four major sections. While specification document formats vary from group to group the format presented here is a good representation of what you will encounter in industrial specifications. The sections of the document are:

1. *Scope*
2. *Applicable Documents*
3. *Engineering Requirements*
4. *Verification of Requirements*

This document is not only intended to explain engineering specification documents and show the development of a specification through the use of a running example. It also aims to help you produce a specification for your project. In order to achieve this goal each section will be presented in three levels. I will:

1. *Attempt to explain the reasons that the section exists and what data goes into that section.*
2. *Complete that section for the running example and annotate the example to make it more understandable.*
3. *Strongly suggest that you complete the section for your project. When you see the symbol*  *it means you are going to get to produce something.*

A skeleton template will be available in the same place that you found this document. Fill out that skeleton template as you follow through this document. You will quickly produce a specification in the least painful way I can think of. Note that I didn't say pain free. Sometimes thinking as hard as you need to think in engineering design just makes your head hurt.



Student Specifications Document Template Guide

Name1

Name2

Name3

August 27, 2019

There is always a fair amount of standard stuff or 'boilerplate' in formal documents. Specifications have a signature page (someone has to approve of the document)...

Signature Page

Signature	*Signature*	*Signature*
Date and email	*Date and email*	*Date and email*

And a revision history (specs do change, the goal is to limit the extent of that change)...

Revision History

Revision	Description	Author	Date	Approval
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

A system example will be used throughout this guide to illustrate the writing of a specification document: an automated system for harvesting saffron. The requirements for its design are in the right column with the narrative in the left column.

From experience I know that I will not get it all right the first time. If you find what you believe to be a mistake or omission, and can make a good case for why you feel that this is so, then I will revise the document and immortalize your name in the revision table.

Contents

1 SCOPE 11

2 APPLICABLE DOCUMENTS 13

3 ENGINEERING REQUIREMENTS 20

4 VERIFICATION OF REQUIREMENTS 23

This section is a very top level, simple verbal description of what the item is and where it is used. See the amplifier example under the **General** *heading.*

This system is a challenging one, but this description is adequate.

Acronyms: *I have not chosen to define acronyms because I have a thing about the overuse of acronyms. You may not be so burdened and find that typing out a name is simply too tedious. Put those acronyms here. Additional short descriptive paragraphs can be added only if needed for special classification, designation of alternate versions or other material that is part of a top-level description.*



Take time to write this section for your project. Being able to write a simple description of even complex things is a good indicator of how well you understand what you are planning.

Specifications

1 SCOPE

General: This document describes the design and verification requirements for a saffron crocus bloom detection and stigma harvesting system.

Background: Saffron is the most expensive spice in the world typically commanding a price of over two thousand US dollars per pound. Seventy thousand blossoms must be harvested in order to produce a pound of the dried spice. Its cultivation is labor intensive for this and other reasons explained below.

The parts of the blossom that are harvested are the three bright orange stigma (called 'threads' by cooks). The stigma are fragile when dry, but flexible when fresh.

The stigma are best harvested shortly after the blossom has opened. The timing of bulb opening is random over the harvest season. This randomness requires that workers constantly monitor the field, picking the open blossom and removing its stigma.

This section is often poorly understood and poorly implemented. It is a list of documents that are a critical part of understanding the item or requirements imposed on the item. Every document listed must have a text reference in the body of the spec further describing and limiting how it is to be applied. Conversely, no document is to be referenced in the spec unless it is listed here. Don't put items here that are background information or of general interest. Always obtain and review all items listed here. This section often has the following statement:

> *The following documents shown shall form part of the specifications for this project. In the event of a conflict between requirements, priority shall first go to the contract, second to this document, and lastly to these reference documents.*

There are lots of MIL-STD(standards) and MIL-HDBK(handbooks) that cover an amazing range of subjects. Here is a website that has them plus NASA documents and others all available (every government specification and handbook you can imagine) for free. Other groups publishing standards include Institute of Electrical and Electronics Engineers (IEEE), American Society of Mechanical Engineers (ASME), Society of Automotive Engineers (SAE), American National Standards Institute (ANSI), American Society for Testing and Materials (ASTM) and Aeronautical Radio, Incorporated (ARINC) to name more than a few.

Another thing to note is that referencing documents will often save you time (and money). For example, a referenced document can contain a complete set of environmental tests. Stating that your system has to be tested to the requirements of MIL-STD xxx is a lot easier than making up the series of tests yourself. The process is akin to what you do when you use a library in C programming: someone has provided software that meets your needs. Why reinvent the wheel? Particularly in something as difficult to get right as environmental testing.

Another benefit to using standards documents is that it connects your project to what is typically done in industry. While this should be obvious it is sometimes forgotten. The designers that you hand the spec over to may already know the standard that you have specified and know how to meet these standards. The simple adherence to standard could save you a lot of money.

My project is pretty simple and I don't have any applicable documents, but your project might. Did you know that in order to use USB on a commercial device that you have to pay a licensing fee? I know, your development board has a USB port on it. You can use it because the board manufacturer paid the fee. The USB spec is about 1500 pages. Saying that your device must comply with this spec (or a part of it) is a lot easier than writing it all out. Think of Applicable Documents like #include files in C. You get all the benefits without having to write all that

2 APPLICABLE DOCUMENTS

(a) **Government Documents** *This is where to put MIL-Specs, MIL-STDs, NASA specs and so forth. Be sure to include the revision level and date.*

(b) **Industry Documents** *This is where to put ANSI, ASTM, ASME, IEEE, Company specifications and so forth. Both this section and government documents can be divided up into logical subcategories.*

code.

 *Research and write down the external specifications that you want your system to meet. Think about standards (like USB) and MIL-Specs that cover things like environmental testing and electromagnetic compatibility.*

Stakeholder Requirements

This section is fundamentally a list of characteristics and features requested by the stakeholders. Stakeholders are anyone that has part in the system whether in designing it, packaging it, using it, maintaining it, etc. Forgetting or ignoring a stakeholder often means change during a later a project phase when change is expensive.

As an example, suppose you are tasked to design a large time/temperature/information display for a bank. The typical bank customer only passively interacts with the device through its display and cannot modify the system through some input, but needs the display to present information in a useful fashion. The bank must be able to set the time and provide the information that they wish their customers to see.
While you, as an electrical or computer engineer, may understand much of the function of the system and how to design it. You may need help with the deign of the power system. You will likely need help packaging the system to withstand the operating environment and to dissipate any heat that is generated by the system. The people that help you with the design are project stakeholders and their needs must be considered while specifying the system.

I have identified the stakeholders in the power amplifier module project listed and they are listed in the specification as follows.

The stakeholders for the saffron crocus stigma plucker are:

1. *The customers (by customer in this context I mean the group requesting the design).*
2. *The consumers (the group that will use the system).*
3. *The packaging/environmental group (they put it in a box and make sure that it can operate in the required environments).*
4. *The mechanical group (they handle moving parts and strength requirements).*
5. *The installation/maintenance group (they install and maintain the system).*
6. *The disposal group (they get rid of it when its useful life is over).*

 *Think about the stakeholders in your project. Write down a list of stakeholders in your project on your specifications template. Think about your project as if it were to be produced commercially. Think deeply.*

Gathering Stakeholder Requirements

One approach to writing client requirements is to use what are called 'user stories'. User stories are essentially the information that a user (stakeholder) will give you when asked what the system must do for them. See the following website for more information on user stories:

(User Stories).

Some users are more difficult to get stories out of than others as these Dilbert cartoons attest.

(Alice Tries to Extract Requirements from a Stakeholder).

(Dilbert Tries the Same).

Requirements answer the 'what' questions about your project.

- *What must it do?*
- *What must it weigh?*
- *What environments must it endure?*
- *What must be done to ensure safety, maintainability, cost, etc.?*
- *What constrains the design (and designers)?*
- *+ many other potential questions in many categories.*

It is important that you don't try and complete the design at this step. We don't design (specify the 'how's (or methods to be a little more precise)) at this point for two primary reasons: we may be only specifying the project and then turning the requirements over to a designer, who like any good designer does not want to be tied down. If we are completing the design ourselves we don't want to pigeon-hole our thinking this early in the design process. Keeping an open mind at this stage of the design often leads us to better thought-out designs. You should marginally note your ideas of how to meet the requirement, but do not require a specific 'how' unless it is truly a design constraint. Don't worry, there will be plenty of time to figure out how we are going to meet the requirements of the project a little later in the design process.

Requirements typically divide themselves into design constraints, functional requirements, and non-functional requirements. For example, a functional requirement would state that the device shall measure and record barometric pressure with such and such accuracy every minute over a 72 hour period. A non-functional requirement would require that the device operates on no more than 500mW average power with power maximums of 1W.

In my opinion, the term 'non-functional requirement' creates a lot of confusion. The term carries shades of 'not required', which is strictly not true. These requirements support the abstract function of or system but they are not the reason we are building the system. I wish we could use the term 'functional support requirement' because it is more descriptive, but with nearly a century of inertia behind the original term my wishes are meaningless. It may, however help you to think 'functional support' as you are writing requirement documents.

We see an example of a true design constraint in the WISE scientific instrument designed by the Space Dynamics Laboratory at Utah State University. The system engineers required a specific FPGA instead of a designer chosen micro-processor simply because they knew that there wasn't any micro that would function in the expected space environment.

Continuing with the saffron crocus stigma plucker as an example, the stakeholder user stories are:

Stakeholders' User Stories

The primary stakeholders needs are described below.
As useful as user stories and user interviews are they are not perfect and your access to users may be incomplete. It is up to you to fill in the missing spots. Some of the areas that are typically missed include power requirements (batteries, wall power, hamsters, etc.), packaging size and weight requirements (standard packaging, wear on your wrist, tow in trailer, or my personal favorite: the portable data acquisition system at Boeing where portable meant you could move it with a fork lift), and operating environments (we live a desert and when we design we often forget humid operating environments).

 *Time to put a name to and then interview your stakeholders! I like this step. It is fun to hear how others who have an interest in this system view it. There is a classic drawing about how different stakeholders see an airplane. You can see the drawing here (How the Stakeholders See the Same Airplane).*

I realize that as a student you may not have access to all of the stakeholders for your project. You may have to role play with your project partners or an available, yet naive, friend. Pretending is fun, at least you used to like it.

3 ENGINEERING REQUIREMENTS

This section constitutes the meat of the document. Remember that it is "requirements" and not methods. The definition here should leave no doubt about what is needed. Engineering requirements differ from stakeholder requirements in measurability, detail, and unambiguity.

Measurability is a key attribute; without it we have no way of knowing that we have built what we said that we would build. In the aerospace industry the process of determining that we have built what we planned to build is called verification. Verification answers the question,

> *"Did we build what we said we would build?"*

Ideally, detail in engineering requirements means that the designer can set off designing and building the system with little or no extra input from the stakeholders. Since we do not live in an ideal world additional stakeholder input will be required. When additional input is sought the specification should be revised to reflect clarification or change. Reality is why we include a Revision History Block at the beginning of the specification.

An ambiguous requirement, by its very nature, means that I could design and build the system in one, two, or many 'wrong' ways. Since the stakeholder who is paying for this system doesn't want one of the many possible wrong ways we need to ensure that requirements cannot be misinterpreted by the designer.

Understand:

1. *"An engineering requirement is a statement about the system that is unambiguous. There's only one way it can be interpreted, and the idea is expressed clearly so all of the stakeholders understand it."*
2. *"An engineering requirement is binding. The customer is willing to pay for it, and will not accept the system without it."*
3. *"An engineering requirement is testable. It's possible to show that the system either does or does not comply with the statement." (Steve Tockey)*

Additionally, requirements must be stated in one place only in this document. You can reference the requirement in another part of the document by using its paragraph number. It is often tempting to restate a requirement in another requirement to improve clarity. However, a requirement stated in two places may only get changed in one place during a revision and possibly leading to confusion or worse a flawed product.

Engineering requirements constitute contacts between between clients and design teams. Requirements are binding.

The section following this one is called Verification. It specifies the methods that the design team need to use to show that the requirements have indeed been met. Writing the verification method sequentially with each requirement forms a good check on how well the requirement is written. If I can't think of a way to show I have met the requirement then the requirement itself may be loose or ambiguous. Paragraph numbering is arranged such that if a requirement has number 3.1.4 then the specified verification method will be described in paragraph 4.1.4.

Requirements form the base for our designs. A well written requirement keeps us from ending up with a less than ideal implementation. Defining requirements forces us to think prior to design. When explaining this concept, I often use the analogy of building a house. We don't build a house by digging a big hole on the lot and pouring cement in it. We would miss all of the important things that have be included in a foundation: water, sewer, electrical, etc. Not to mention that there are rules that we have to follow in order to build a house legally. We study codes, make calculations, and draw pictures. We work hard long before we dig the hole and pour the cement. We save time and money by planning. Can you imagine the cost of having to cut up a basement floor in order to install sewer lines?

The engineer's job at this point is to make the stakeholder requirements of the previous section into measurable, detailed, and unambiguous. Sometimes this process requires that a stakeholder be interviewed again until a proper set of engineering requirements emerge.

An old joke in aerospace says that a specification is written and thrown over a high wall to the design group. The specification writer then runs away from the wall (laughing). While formal adoption of this absurd method leads to disaster, we should write specifications as if we could never clarify anything. The clearer we make the specification now, the lower the cost of change later.

4 VERIFICATION OF REQUIREMENTS

Were the requirements met? This process of determining this is called 'verification' and it answers the fundamental question: "Did we build what was required?". We verify requirements by testing to see if the requirements are met. Design teams must test every requirement in order to prove that the requirement is met. Testing each requirement means that each requirement must be cast in some quantifiable way. In this section you specify how you will test each requirement to show that it has been met.

Don't worry! It is not as bad as it sounds. Sometimes (not always!) you can verify or test a requirement simply by looking at the completed system to make sure some required thing is present.

Testing often 'takes it on the chin' in terms of project schedule. Since integrated system testing typically occurs near the end of a project, the time for testing is compressed against the deadline. People start short-cutting tests to stay on schedule. Sometimes you may get away with it but it is never a good idea either technically or ethically. Epic failures have occurred because of truncated testing. One such failure occurred during the testing of the Hubble Space Telescope. The following is an excerpt from the the official report detailing the failure.

> *Reliance on a single test method was a process which was clearly vulnerable to simple error. Such errors had been seen in other telescope programs, yet no independent tests were planned, although some simple tests to protect against major error were considered and rejected. During the critical time period, there was great concern about cost and schedule, which further inhibited consideration of independent tests.*
>
> *The Hubble Space Telescope Optical Systems Failure Report-NASA November 1990*

If you are interested the whole report is available at (`https://www.ssl.berkeley.edu/~mlampton/AllenReportHST.pdf` *).*

The Hubble error wasn't caught until the telescope was deployed in space. Can you imagine the cost of fixing this problem? It is not simply a case of bundling you off with your instruments and putting you up in a fancy hotel for a week or two. Some estimates set the price at about $1 billion.

The Dilbert comic strip has a similar, and darkly amusing, view of testing truncation.

(Ship It!)

(Communism)

Possible verification methods include:

1. *Inspection:*
   Inspection is a method of verification consisting of investigation, without the use of special laboratory appliances or procedures, to determine compliance with requirements. Inspection is generally nondestructive and includes (but is not limited to) visual examination, manipulation, gauging, and measurement.

2. *Demonstration:*
   Demonstration is a method of verification that is limited to readily observable functional operation to determine compliance with requirements. This method shall not require the use of special equipment or sophisticated instrumentation.

3. *Analysis:*
   Analysis is a method of verification, taking the form of the processing of accumulated results and conclusions, intended to provide proof that verification of a requirement has been accomplished. The analytical results may be based on engineering study, compilation or interpretation of existing information, similarity to previously verified requirements, or derived from lower level examinations, tests, demonstrations, or analyses.

4. *Direct Test:*

   Test is a method of verification that employs technical means, including (but not limited to) the evaluation of functional characteristics by use of special equipment or instrumentation, simulation techniques, and the application of established principles and procedures to determine compliance with requirements.

 *Now read over your completed specification and make additions and corrections. Find others who will be willing to read and comment on the specification (hopefully they will still like you when they are done). The more eyes the better. Ask yourself if you handed this specification to a competent design team what would they build?*

Congratulations! You have written an engineering specification and that is no mean feat.